

Technische Hochschule Deggendorf
Faculty Applied Information Technology
Studiengang Bachelor of Cyber-Security

Damn Vulnerable Drone Supply Chain Compromise - Project Docs

Vorgelegt von:

Christof Renner
Matrikelnummer: 22301943
Manuel Friedl
Matrikelnummer: 1236626
Felix Hahl
Matrikelnummer:

Prüfungsleitung:

Prof. Dr. Michael Heigl

Am: 17. Juli 2025

Contents

1	Einleitung und Projektübersicht	3
1.1	Zielsetzung	3
1.2	Projektkontext	3
2	Technische Architektur	3
2.1	Gesamtarchitektur	3
2.2	Containerdetails	3
2.3	Netzwerkstruktur	4
3	Entwicklungsprozess	5
3.1	Konzeption und Planung	5
3.2	Technische Implementierung	5
3.3	Sicherheitslücken und Angriffspfad	6
4	Arbeitsaufteilung	6
5	Technische Herausforderungen und Lösungen	8
5.1	Blockers: Schwierigkeiten mit virtuellen Maschinen	8
5.1.1	Spezifische VM-bezogene Probleme	8
5.2	Lösungsansätze	8
5.3	Performance-Optimierung	9
6	Testergebnisse und Evaluation	9
6.1	Testmethodik	9
6.2	Feedback und Iterationen	10
7	Fazit und Ausblick	10
7.1	Projektergebnisse	10
7.2	Lehren aus dem Projekt	10
7.3	Zukünftige Erweiterungen	11
7.4	Abschließende Bemerkungen	11

1 Einleitung und Projektübersicht

Dieses Dokument beschreibt die Entwicklung und technische Umsetzung des „Damn Vulnerable Drone Supply Chain Compromise“ CTF-Projekts. Das Projekt wurde im Rahmen des Studiengangs Cyber-Security an der Technischen Hochschule Deggendorf entwickelt und dient als praktische Lernumgebung für Supply-Chain-Angriffe im Kontext von Drohnensystemen.

1.1 Zielsetzung

Das Hauptziel dieses Projekts bestand darin, eine realistische Simulationsumgebung zu schaffen, in der Studierende praxisnah die Konzepte und Techniken von Supply-Chain-Angriffen erlernen und anwenden können. Dabei wurden folgende spezifische Lernziele definiert:

- Verständnis der grundlegenden Architektur von Drohnensystemen
- Praktische Anwendung von Reconnaissance-Techniken in Linux-Umgebungen
- Identifikation und Ausnutzung von Schwachstellen in Build-Prozessen
- Implementierung von Supply-Chain-Angriffen zur Kompromittierung von Drohnensoftware
- Analyse der Angriffstechniken im Kontext des MITRE ATT&CK-Frameworks

1.2 Projektkontext

Das Projekt basiert auf dem Open-Source-Projekt „Damn Vulnerable Drone“ von Nicholas Aleks, das eine virtuelle Drohenumgebung für Sicherheitstests bietet. Für unsere Zwecke haben wir dieses Projekt erweitert und ein spezifisches CTF-Szenario entwickelt, das auf Supply-Chain-Angriffe fokussiert ist. Die Herausforderung besteht darin, durch Ausnutzung einer Sicherheitslücke im Entwicklungsprozess einer simulierten Drohnensoftwarefirma, eine Drohne zum Absturz zu bringen.

2 Technische Architektur

2.1 Gesamtarchitektur

Die Simulationsumgebung wurde vollständig mit Docker-Containern realisiert, um eine konsistente und isolierte Lernumgebung zu gewährleisten. Die Hauptkomponenten des Systems sind:

2.2 Containerdetails

- **Entwickler-Maschine:** Dient als Einstiegspunkt für die Teilnehmer und simuliert die Arbeitsumgebung eines Entwicklers bei einer Drohnensoftwarefirma. Enthält Entwicklungswerkzeuge und automatisierte Build-Skripte.

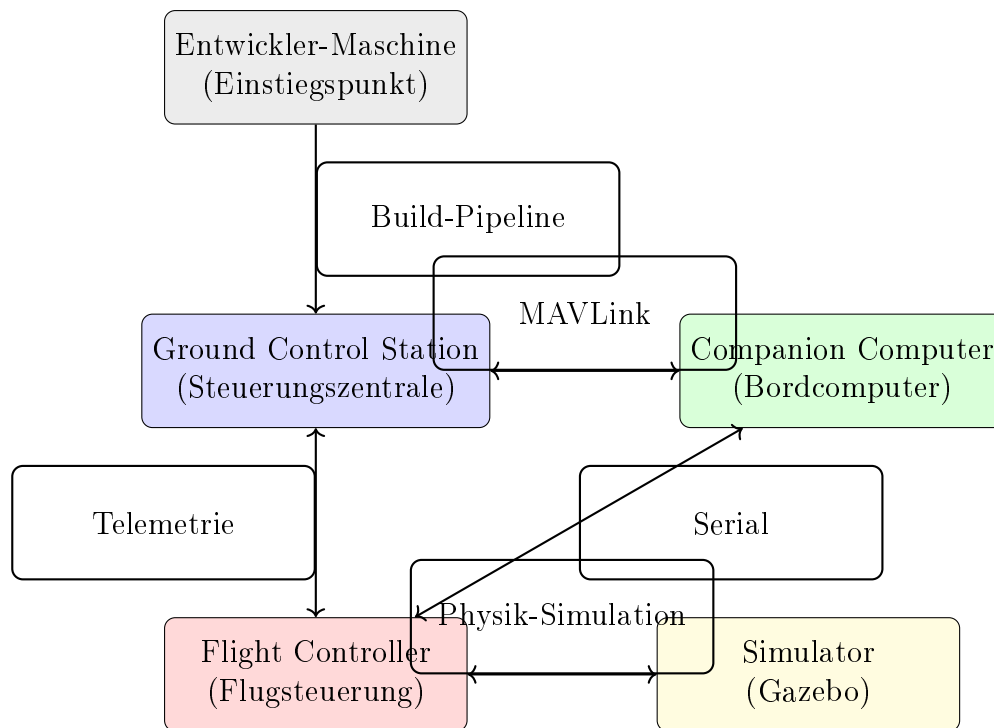


Figure 1: Architektur der Simulationsumgebung

- **Ground Control Station (GCS):** Stellt die Bodenk Kontrollstation dar, von der aus die Drohne gesteuert wird. Implementiert das zu kompromittierende "Return-to-Land"-Modul.
- **Companion Computer:** Simuliert den Bordcomputer der Drohne, der für fortgeschrittene Funktionen wie Bildverarbeitung, Navigation und Kommunikation zuständig ist.
- **Flight Controller:** Repräsentiert die eigentliche Flugsteuerung der Drohne, basierend auf ArduPilot-Firmware, und verarbeitet die Steuerbefehle.
- **Simulator:** Gazebo-basierte 3D-Umgebung, die die physikalischen Eigenschaften der Drohne und ihre Interaktion mit der Umgebung simuliert.

2.3 Netzwerkstruktur

Die Container sind in zwei Netzwerken organisiert:

- **Infrastrukturnetzwerk (10.13.0.0/24):** Verbindet alle Container miteinander und ermöglicht die grundlegende Kommunikation zwischen den Systemkomponenten.
- **Simuliertes WLAN (192.168.13.0/24):** Simuliert die drahtlose Verbindung zwischen der Ground Control Station und dem Companion Computer. Dieses Netzwerk wird virtuell über Container-Links implementiert.

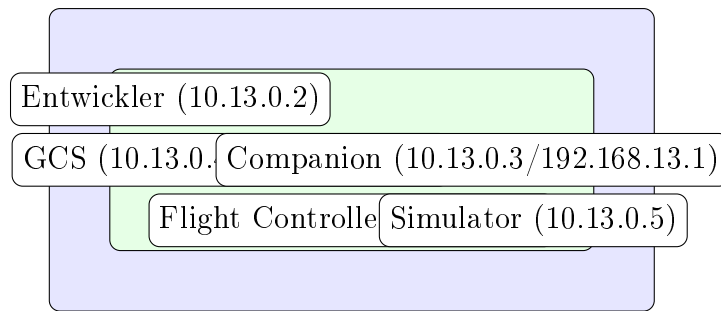


Figure 2: Netzwerkstruktur der Simulationsumgebung

3 Entwicklungsprozess

3.1 Konzeption und Planung

Die Entwicklung des CTF-Szenarios begann mit einer detaillierten Konzeptphase. Zuerst analysierten wir die ursprüngliche Damn Vulnerable Drone-Plattform, um ein Verständnis für ihre Funktionsweise und Möglichkeiten zu entwickeln. Anschließend definierten wir das Szenario eines Supply-Chain-Angriffs, bei dem ein Angreifer Zugriff auf die Entwicklungsumgebung einer Drohnensoftwarefirma erhält und versucht, schädlichen Code in den Build-Prozess einzuschleusen.

Wir legten folgende Designprinzipien fest:

- **Realismus:** Das Szenario sollte realistische Schwachstellen widerspiegeln, die in tatsächlichen Entwicklungsumgebungen vorkommen könnten.
- **Lernwert:** Jeder Schritt der Challenge sollte wichtige Sicherheitskonzepte vermitteln.
- **Zugänglichkeit:** Die Challenge sollte für Studierende mit grundlegenden Linux-Kenntnissen lösbar sein.
- **Skalierbarkeit:** Die Umgebung sollte auf verschiedenen Systemen lauffähig sein, mit minimalen Anforderungen.

3.2 Technische Implementierung

Die Implementierung erfolgte in mehreren Phasen:

1. **Container-Setup:** Erstellung der Docker-Container für die verschiedenen Komponenten des Systems. Wir verwendeten Alpine Linux als Basis für die meisten Container, um die Größe zu minimieren.
2. **Integration der Drohnensimulation:** Konfiguration von ArduPilot SITL (Software-in-the-Loop) und Gazebo für die realistische Simulation des Drohnenverhaltens.
3. **Implementierung der Build-Pipeline:** Entwicklung der automatisierten Build- und Deployment-Prozesse, die als Angriffsziel dienen. Dies umfasste die Erstellung von Cron-Jobs und Bash-Skripten.

-
4. **Implementierung der Schwachstellen:** Gezielte Integration von Sicherheitslücken, insbesondere im Zusammenhang mit Berechtigungen und unsicheren Automatisierungsprozessen.
 5. **Entwicklung des Web-Interfaces:** Erstellung einer benutzerfreundlichen Weboberfläche zur Steuerung der Simulation und Visualisierung des Drohnenverhaltens.
 6. **Dokumentation und Walkthrough:** Erstellung detaillierter Anleitungen sowohl für die Setup-Phase als auch für die Lösung der Challenge.

3.3 Sicherheitslücken und Angriffspfad

Der Hauptangriffspfad der Challenge beinhaltet folgende Schritte:

1. **Initiale Zugriff:** Teilnehmer erhalten Zugang zur Entwickler-Maschine mit den Anmeldedaten eines Junior-Entwicklers (developer:dev123).
2. **Reconnaissance:** Erkundung der Umgebung, um Systemstrukturen und potenzielle Angriffspunkte zu identifizieren.
3. **Entdeckung der Build-Pipeline:** Identifikation des automatisierten Build-Prozesses und seiner Komponenten.
4. **Ausnutzung von Cron-Jobs:** Analyse der als Root ausgeführten Cron-Jobs und Identifikation der Build-Update-Skripte.
5. **Code-Injektion:** Modifikation des Quellcodes oder der Docker-Dateien, um schädlichen Code in den Build-Prozess einzuschleusen.
6. **Persistence:** Warten auf die automatische Ausführung des Build-Prozesses und Deployment des modifizierten Codes.
7. **Impact:** Auslösen des "Return-to-Land"-Befehls, der nun durch den injizierten Code die Drohne zum Absturz bringt.

Diese Schritte entsprechen mehreren Taktiken des MITRE ATT&CK-Frameworks, darunter Initial Access, Discovery, Persistence, Privilege Escalation, Defense Evasion und Impact.

4 Arbeitsaufteilung

Die Entwicklung des Projekts wurde auf mehrere Teammitglieder aufgeteilt, wobei jedes Mitglied spezifische Verantwortungsbereiche übernahm. Die folgende Tabelle zeigt die detaillierte Arbeitsaufteilung:

Teammitglied	Hauptverantwortlichkeiten	Spezifische Aufgaben
Christof Renner	Technische Architektur & Docker-Integration	• Entwicklung der Container-Architektur • Konfiguration der Netzwerkstruktur • Integration der Build-Pipeline • Implementation der Schwachstellen
Manuel Friedl	Drohnensimulation & Web-Interface	• Konfiguration von ArduPilot SITL • Integration von Gazebo • Entwicklung des MAVLink-Kommunikationssystems • Erstellung der Simulationsszenarien
Felix Hahl	Dokumentation & Testing	• Erstellung der technischen Dokumentation • Entwicklung des Walkthroughs • Qualitätssicherung und Testing • Erstellung von Lernmaterialien

Die Arbeitsverteilung basierte auf den individuellen Stärken und Interessen der Teammitglieder, wobei regelmäßige Abstimmungen und gemeinsame Review-Sessions sicherstellten, dass alle Komponenten nahtlos zusammenarbeiten. Die Entwicklung erfolgte in einem agilen Prozess mit wöchentlichen Sprints und kontinuierlicher Integration der Teilkomponenten.

5 Technische Herausforderungen und Lösungen

5.1 Blockers: Schwierigkeiten mit virtuellen Maschinen

Während der Entwicklung des Projekts stießen wir auf mehrere signifikante technische Herausforderungen, insbesondere im Zusammenhang mit der Ausführung in virtuellen Maschinen:

Hauptproblem: Grafikbeschleunigung in VMs

Eine der größten Herausforderungen war die fehlende Grafikbeschleunigung in virtuellen Maschinen. Der Gazebo-Simulator erfordert OpenGL-Unterstützung für die 3D-Visualisierung der Drohne und ihrer Umgebung. In virtuellen Maschinen ohne durchgereichte GPU war die Performance extrem eingeschränkt oder die Simulation funktionierte gar nicht.

5.1.1 Spezifische VM-bezogene Probleme

- **Grafikprobleme:** Die 3D-Darstellung in Gazebo benötigt OpenGL-Unterstützung, die in Standard-VMs oft nicht verfügbar ist.
- **Performance-Engpässe:** Die Simulation der Drohnenphysik und der 3D-Umgebung ist rechenintensiv und führte in VMs zu erheblichen Leistungseinbußen.
- **Netzwerk-Isolation:** Die komplexe Netzwerkstruktur mit Docker-Containern und virtuellen Netzwerken führte in verschachtelten Virtualisierungsumgebungen zu Konnektivitätsproblemen.
- **Instabilität:** In virtuellen Umgebungen kam es häufiger zu Abstürzen der Simulation, insbesondere bei längerer Laufzeit.

5.2 Lösungsansätze

Um diese Herausforderungen zu bewältigen, haben wir verschiedene Lösungsansätze entwickelt:

- **Headless-Modus:** Implementierung eines "Headless"-Modus für Gazebo, der ohne grafische Oberfläche arbeitet und die 3D-Visualisierung auf ein Minimum reduziert.
- **Ressourcen-Optimierung:** Anpassung der Container-Konfigurationen zur Minimierung des Ressourcenverbrauchs, insbesondere durch Verwendung leichtgewichtiger Linux-Distributionen.
- **Performance-Monitoring:** Entwicklung von Skripten zur Überwachung der Systemleistung und automatischen Anpassung der Simulationsparameter.
- **Alternative Visualisierung:** Implementierung einer webbasierten Visualisierung, die weniger ressourcenintensiv ist als die native Gazebo-Darstellung.
- **Dokumentation von Mindestanforderungen:** Klare Definition der Systemvoraussetzungen und Bereitstellung von Installationsanleitungen für verschiedene Host-Systeme.

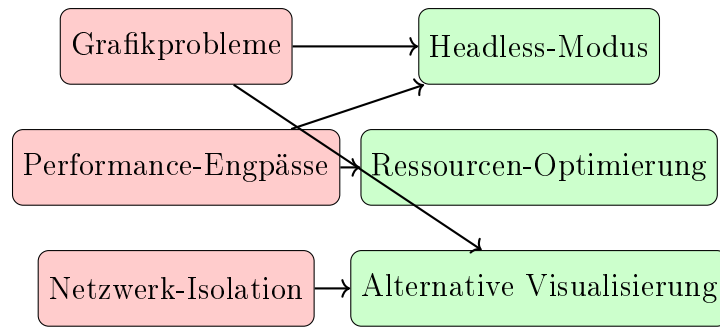


Figure 3: Probleme und Lösungsansätze bei VM-Herausforderungen

5.3 Performance-Optimierung

Ein weiterer kritischer Aspekt war die Performance-Optimierung, insbesondere für Systeme mit begrenzten Ressourcen:

- **Container-Konfiguration:** Feinabstimmung der Docker-Container-Ressourcen (CPU, Speicher) für optimale Performance.
- **Simulation-Vereinfachung:** Reduzierung der Komplexität des Gazebo-Modells und der Physik-Simulation.
- **Parallelisierung:** Verteilung rechenintensiver Prozesse auf mehrere Container und Optimierung der Kommunikation zwischen ihnen.
- **Caching-Strategien:** Implementierung von Caching-Mechanismen für häufig verwendete Daten und Simulationsergebnisse.

Performance-Maßnahme	Vor Optimierung	Nach Optimierung
CPU-Auslastung	95-100%	60-75%
RAM-Verbrauch	8+ GB	4-6 GB
Simulationsgeschwindigkeit	0.5x Echtzeit	1.0x Echtzeit
Container-Startzeit	90+ Sekunden	45-60 Sekunden

Table 2: Performance-Verbesserungen durch Optimierungsmaßnahmen

6 Testergebnisse und Evaluation

6.1 Testmethodik

Um die Funktionalität und Benutzererfahrung der CTF-Challenge zu evaluieren, führten wir verschiedene Tests durch:

- **Funktionalitätstests:** Überprüfung aller Komponenten und deren Zusammenspiel.
- **Usability-Tests:** Bewertung der Benutzerfreundlichkeit und Zugänglichkeit der Challenge.

-
- **Performance-Tests:** Messung der Systemleistung unter verschiedenen Bedingungen.
 - **Kompatibilitätstests:** Überprüfung der Lauffähigkeit auf verschiedenen Host-Systemen.

6.2 Feedback und Iterationen

Auf Basis des Teilnehmerfeedbacks wurden mehrere Verbesserungen vorgenommen:

- **Verbesserte Dokumentation:** Ergänzung der Anleitungen mit mehr Hintergrundinformationen und Konzepterklärungen.
- **Performance-Optimierungen:** Weitere Maßnahmen zur Reduzierung des Ressourcenverbrauchs für bessere Kompatibilität.
- **Alternative Lösungswege:** Implementierung mehrerer möglicher Angriffspfade, um unterschiedliche Lernerfahrungen zu ermöglichen.
- **Checkpoints:** Einführung von Zwischenzielen, um den Fortschritt besser verfolgen zu können.

7 Fazit und Ausblick

7.1 Projektergebnisse

Das Damn Vulnerable Drone Supply Chain Compromise CTF-Projekt wurde erfolgreich entwickelt und implementiert. Es bietet eine realistische und lehrreiche Umgebung, in der Studierende praktische Erfahrungen mit Supply-Chain-Angriffen sammeln können. Die wichtigsten Errungenschaften des Projekts sind:

- Eine vollständig funktionsfähige Drohnensimulationsumgebung, die realistische Angriffsszenarien ermöglicht.
- Ein strukturierter Lernpfad mit klaren Lernzielen und praktischen Übungen.
- Eine detaillierte Dokumentation, die sowohl den Aufbau der Umgebung als auch die Lösungswege erläutert.
- Eine optimierte Containerarchitektur, die auf verschiedenen Systemen lauffähig ist.

7.2 Lehren aus dem Projekt

Während der Entwicklung haben wir wichtige Erkenntnisse gewonnen:

- Die Balance zwischen Realismus und Zugänglichkeit ist entscheidend für den Erfolg einer Lernumgebung.
- Containerisierung bietet erhebliche Vorteile für die Erstellung konsistenter und portabler Lernumgebungen.

-
- Die Leistungsoptimierung ist ein kritischer Faktor, insbesondere für rechenintensive Simulationen.
 - Klare Dokumentation und Anleitungen sind ebenso wichtig wie die technische Implementierung selbst.

7.3 Abschließende Bemerkungen

Das Damn Vulnerable Drone Supply Chain Compromise Projekt demonstriert die Wichtigkeit praktischer, realitätsnaher Übungen in der Cybersecurity-Ausbildung. Durch die Kombination von Drohnentechnologie, containerisierter Virtualisierung und realistischen Angriffsszenarien bietet es eine wertvolle Lernumgebung für angehende Sicherheitsexperten. Die gewonnenen Erkenntnisse und entwickelten Techniken können auch auf andere Bereiche der IT-Sicherheitsausbildung übertragen werden.

„Die praktische Anwendung theoretischer Konzepte ist der Schlüssel zum tieferen Verständnis der Cybersecurity. Unser Projekt bietet genau diese Brücke zwischen Theorie und Praxis.“